

Software installation with ROS2

In this tutorial, we will learn how to install ROS2 Jazzy Jalisco on Ubuntu 24.04 and build our software stack.

TLDR: single command setup

Prerequisites - You have a running Ubuntu 24.04 environment - You have an existing Github account and added a SSH key to your account - You have root access to your system (sudo)

If you have not previously set up any of our software stack, you can use the following command to install and setup everything in one go:

```
mkdir -p ~/git/bitbots \
  && cd ~/git/bitbots \
  && curl -fsSL https://raw.githubusercontent.com/bit-bots/bitbots_main/main/scripts/
setup.sh > /tmp/setup.sh \
  && bash /tmp/setup.sh
```

Manual steps with in depth explanation

0. Use Ubuntu 24.04

As ROS works best on Ubuntu, we are using this distribution. Currently, ROS2 Jazzy runs on Ubuntu 24.04.

If you are not already using Ubuntu 24.04, consider installing it on your system (perhaps as a dual boot?). Alternatively you can use a devcontainer [VSCode Dev Container Setup](#), with a preconfigured environment and follow those instructions, as these docs do not apply to the devcontainer.

1. Setup and Install ROS 2

- Follow this guide and when it comes to the section **Install ROS 2 packages**, install the recommended `ros-jazzy-desktop-full`: <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

- Install additional dependencies:

```
sudo apt install \  
  clang-format \  
  cppcheck \  
  python3-colcon-clean \  
  python3-colcon-common-extensions \  
  python3-pip \  
  python3-rosdep \  
  python3-vcstool \  
  ros-jazzy-plotjuggler-ros \  
  ros-jazzy-rmw-cyclonedds-cpp \  
  ros-jazzy-rqt-robot-monitor \  
  ros-jazzy-rqt-runtime-monitor
```

- Run `sudo rosdep init` to initialize `rosdep`, a tool that helps you install system dependencies for ROS packages.

2. Download our software (if not already done)

- Create a GitHub account, if not already done (see [here](#) for further information)
- **Add your SSH key to GitHub to access and sync our repositories**
 - If you don't know what I am talking about or you don't yet have a SSH key, follow this guide: <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/checking-for-existing-ssh-keys>
 - Go to your account settings and add your SSH key (the `.pub` file) to [GitHub](#)
- **Now, you can clone (download) our main code repository (repo) called `bitbots_main`:**
 - **Open a terminal and go to the directory where you want to download our code (typically `~/git/bitbots/`)**
 - Create the directory with: `mkdir -p ~/git/bitbots` This is where your source code will live and grow.
 - Move to this directory with: `cd ~/git/bitbots`
 - Clone the code repository with: `git clone git@github.com:bit-bots/bitbots_main.git`
Confirm the host key by typing `yes`, if asked.
 - Move into the newly created directory with: `cd bitbots_main`
 - Clone all code and other files by running: `make install` This will take a while, as it downloads all the code and other files from our repositories and additionally installs all missing dependencies (using `rosdep` and `pip`). Finally, it will register pre-commit hooks (automatic code-formatting and warnings), which will be run every time you commit code to our repositories.

3. Install Webots

Webots is a robot simulator, which we use to simulate our robots and test our software. It is not strictly necessary to install it, but it is very useful for development and testing. If you want to install it, you can do so by running `make webots` in the `bitbots_main` repository.

4. Setup colcon workspace

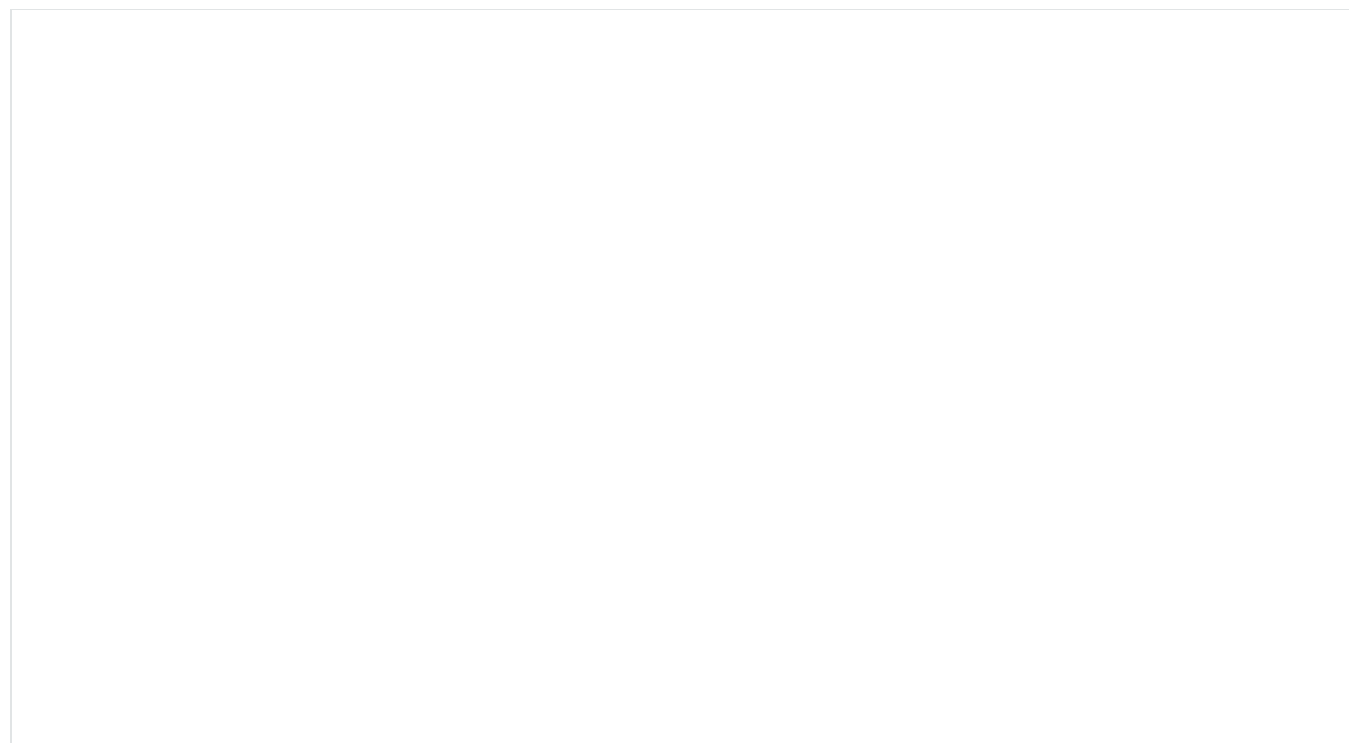
`Colcon` is the tool provided by ROS 2 to build and install our ROS packages, so that they can be launched later. The colcon workspace is where your source code gets build and where we use colcon.

- Create colcon workspace directory (typically `~/colcon_ws/`)
 - Create directory with: `mkdir -p ~/colcon_ws/src`
 - Link our software contained in the `bitbots_main` repo to the newly created `src` directory with: `ln -s ~/git/bitbots/bitbots_main/ ~/colcon_ws/src/bitbots_main`

5. Final touches

To let your system know where it should find all the ROS 2 dependencies and packages and to add colored output etc., we add a little bit of config to your `~/.bashrc` file, which will be run every time you open a new terminal. In case you are not using the bash shell, replace `~/.bashrc` and `bash` with your shell's configuration file.

- Run the following command:



```
cat >> ~/.bashrc << EOF

# >>> bit-bots initialize >>>

# Add python pip bins to PATH
export PATH="\$HOME/.local/bin:\$PATH"

# Ignore some deprecation warnings
export
PYTHONWARNINGS="ignore::setuptools.command.install,ignore::setuptools.command.easy_install
command is deprecated,ignore:setup.py install is deprecated"

# Limit ROS 2 communication to localhost (can be overridden when needed)
export ROS_DOMAIN_ID=24
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST

# Set the default colcon workspace
export COLCON_WS="\$HOME/colcon_ws"

# Set the default log level for colcon
export COLCON_LOG_LEVEL=30

# Define a log layout
export RCUTILS_COLORIZED_OUTPUT=1
export RCUTILS_CONSOLE_OUTPUT_FORMAT="[severity] [name]: {message}"

# Set the default Middleware
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# Load our ros plugin script containing useful functions and aliases for ROS 2 development
if [[ -f \$COLCON_WS/src/bitbots_main/scripts/ros.plugin.sh ]]; then
    source \$COLCON_WS/src/bitbots_main/scripts/ros.plugin.sh
fi

# <<< bit-bots initialize <<<

EOF

source ~/.bashrc
```

- Configure the robot hostnames, see [Configure host names](#).

Notes

Custom docker setup

Before utilizing a devcontainer, we used a custom docker setup for ROS 2 development. If you want (or need) to utilize a custom setup like this, have a look at <https://github.com/timonegk/rosdocked>.

Virtual Machine setup

We recommend against using a virtual machine for ROS 2 development, both for compile

